

US Patent & Trademark Office

Patent Public Search | Text View

United States Patent Application Publication

20250280115

Kind Code

A1

Publication Date

September 04, 2025

Inventor(s)

Kalva; Hari et al.

METHODS, SYSTEMS AND DECODER FOR COMBINED LOSSLESS AND LOSSY CODING

Abstract

A decoder includes circuitry configured to receive an encoded bitstream, select a current frame, identify a first sub-picture of the current frame encoded using a lossless encoding protocol, and decode the current frame, wherein decoding the current frame includes decoding the first sub-picture using the lossless encoding protocol.

Inventors: Kalva; Hari (Boca Raton, FL), Furht; Borivoje (Boca Raton, FL), Adzic; Velibor (Boca Raton, FL)

Applicant: OP Solutions LLC (Amherst, MA)

Family ID: 78293533

Assignee: OP Solutions LLC (Amherst, MA)

Appl. No.: 19/213081

Filed: May 20, 2025

Related U.S. Application Data

parent US continuation 18385000 20231030 parent-grant-document US 12316833 child US 19213081

parent US continuation 17245073 20210430 parent-grant-document US 11889055 child US 18385000

us-provisional-application US 63018629 20200501

Publication Classification

Int. Cl.: H04N19/105 (20140101); H04N19/169 (20140101); H04N19/176 (20140101); H04N19/90 (20140101)

U.S. Cl.:

CPC H04N19/105 (20141101); **H04N19/176** (20141101); **H04N19/188** (20141101);
H04N19/90 (20141101);

Background/Summary

CROSS-REFERENCE TO RELATED APPLICATIONS [0001] This application is a continuation of copending application Ser. No. 18/385,000, filed on Oct. 30, 2023, and titled METHODS, SYSTEMS AND ENCODED BITSTREAM FOR COMBINED LOSSLESS AND LOSSY CODING, which application is a continuation of copending application Ser. No. 17/245,073 filed on Apr. 30, 2021, and titled METHODS AND SYSTEMS FOR COMBINED LOSSLESS AND LOSSY CODING, which application claims the benefit of priority of U.S. Provisional Patent Application Ser. No. 63/018,629, filed on May 1, 2020, and titled “METHODS AND SYSTEMS FOR COMBINED LOSSLESS AND LOSSY CODING,” each of which is hereby incorporated by reference herein in its entirety.

FIELD OF THE INVENTION

[0002] The present invention generally relates to the field of video compression. In particular, the present invention is directed to methods and systems for combined lossless and lossy coding.

BACKGROUND

[0003] A video codec can include an electronic circuit or software that compresses or decompresses digital video. It can convert uncompressed video to a compressed format or vice versa. In the context of video compression, a device that compresses video (and/or performs some function thereof) can typically be called an encoder, and a device that decompresses video (and/or performs some function thereof) can be called a decoder.

[0004] A format of the compressed data can conform to a standard video compression specification. The compression can be lossy in that the compressed video lacks some information present in the original video. A consequence of this can include that decompressed video can have lower quality than the original uncompressed video because there is insufficient information to accurately reconstruct the original video.

[0005] There can be complex relationships between the video quality, the amount of data used to represent the video (e.g., determined by the bit rate), the complexity of the encoding and decoding algorithms, sensitivity to data losses and errors, ease of editing, random access, end-to-end delay (e.g., latency), and the like.

[0006] Motion compensation can include an approach to predict a video frame or a portion thereof given a reference frame, such as previous and/or future frames, by accounting for motion of the camera and/or objects in the video. It can be employed in the encoding and decoding of video data for video compression, for example in the encoding and decoding using the Motion Picture Experts Group (MPEG)'s advanced video coding (AVC) standard (also referred to as H.264). Motion compensation can describe a picture in terms of the transformation of a reference picture to the current picture. The reference picture can be previous in time when compared to the current picture, from the future when compared to the current picture. When images can be accurately synthesized from previously transmitted and/or stored images, compression efficiency can be improved.

SUMMARY OF THE DISCLOSURE

[0007] In an aspect, an encoder includes circuitry configured to receive an input video, select a current frame, identify a first region of the current frame to be encoded using a first lossless encoding protocol, and encode the current frame, wherein encoding the current frame further

comprises encoding the first region using the first lossless encoding protocol.

[0008] A method of combined lossless and lossy coding includes receiving, by an encoder, an input video, selecting, by the encoder, a current frame, identifying, by the encoder, a first region of the current frame to be encoded using a first lossless encoding protocol, and encoding, by the encoder, the current frame, wherein encoding the current frame further comprises encoding the first region using the first lossless encoding protocol.

[0009] These and other aspects and features of non-limiting embodiments of the present invention will become apparent to those skilled in the art upon review of the following description of specific non-limiting embodiments of the invention in conjunction with the accompanying drawings.

Description

BRIEF DESCRIPTION OF THE DRAWINGS

[0010] For the purpose of illustrating the invention, the drawings show aspects of one or more embodiments of the invention. However, it should be understood that the present invention is not limited to the precise arrangements and instrumentalities shown in the drawings, wherein:

[0011] FIG. 1 is a block diagram illustrating an exemplary embodiment of a frame having a plurality of sub-pictures;

[0012] FIG. 2 is an illustration of an exemplary embodiment of a frame having two sub-pictures;

[0013] FIG. 3 is a process flow diagram illustrating an example process for decoding a video according to some implementations of the current subject matter;

[0014] FIG. 4 is a system block diagram illustrating an example decoder capable of decoding a bit stream according to some implementations of the current subject matter;

[0015] FIG. 5 is a process flow diagram illustrating an example process for encoding a video according to some implementations of the current subject matter;

[0016] FIG. 6 is a process flow diagram illustrating an example process of encoding a video according to some implementations of the current subject matter; and

[0017] FIG. 7 is a block diagram of a computing system that can be used to implement any one or more of the methodologies disclosed herein and any one or more portions thereof.

[0018] The drawings are not necessarily to scale and may be illustrated by phantom lines, diagrammatic representations and fragmentary views. In certain instances, details that are not necessary for an understanding of the embodiments or that render other details difficult to perceive may have been omitted.

DETAILED DESCRIPTION

[0019] In traditional video coding systems, video sequence is divided into groups-of-pictures (GOP). Each GOP is self-contained in the sense of temporal and spatial prediction. Usually, first picture in the group is used as a reference picture for the subsequent pictures. Temporal and spatial relationships between the pictures allow for very efficient compression using predictive coding.

[0020] Past coding systems have typically operated using lossy coding, in which some information from an encoded frame is omitted during the encoding process and is not recovered during decoding. Such lossy processes may sacrifice a certain degree of detail and/or resolution in decoded frames and/or video pictures to achieve higher degrees of efficiency, for instance and without limitation by reducing quantities of data transmitted in a bit stream from an encoder to a decoder, processing time and/or memory resources used to encode and/or decode a frame or group of pictures, or the like.

[0021] An alternative approach to the above process may include lossless encoding, wherein a frame is encoded and decoded with no or negligible loss of information; this may result in greater resolution and/or other detail in an output frame and/or video picture. However, while lossless encoding and decoding may occasionally be more efficient for certain kinds of image processing as

noted in further detail below, lossless encoding can also be very expensive in terms of memory resources and processing times. This is particularly apparent in ultra high definition (UHD) video coding, in which a picture or image size may go up to 8K×4K (7680×4320); a big picture size may pose great challenge for chip and/or module design. One reason for this is that the UHD requires a bigger search range in motion estimation and on-chip or other processing memory for buffering reference blocks for motion estimation and compensation. UHD processing may even present challenges for lossy encoding and decoding owing to the greater picture sizes involved.

[0022] Embodiments disclosed herein enable more efficient signaling, decoding, and encoding using combined lossless and lossy video compression coding. In an embodiment, a picture may first be divided into sub-pictures based on quality and computation requirements. An encoder may create as many sub-pictures as there are processing cores (or hardware threads) on a CPU or other device, circuit, or component that is performing encoding and/or decoding of pictures and/or GOP. Since each sub-picture may be independently coded, this form of task partitioning may allow for efficient encoding and/or decoding by using all available computing resources effectively.

Moreover, lossless encoding may furnish better compression than lossy coding that uses transform and quantization, for instance for certain sub-pictures of an overall frame; as a result, combined lossless and lossy coding may result in superior performance to lossless coding alone.

[0023] In some implementations, picture segmentation is performed using sample blocks as a basic unit. Sample blocks may have a uniform sample block size, which may a side length, in pixels of a square of pixels; for instance and without limitation, embodiments disclosed herein may use 4×4 sample blocks as a basic unit and the measure of spatial activity that includes computing frequency components of the 4×4 sample blocks. By utilizing 4×4 blocks as a basic segmentation size some implementations of the current subject matter may allow for finer granularity by the encoder and such a size aligns with established transform block sizes, enabling utilizing of standard and defined transform matrices, which may improve encoding efficiency. Further, such an approach may be contrasted with some existing approaches to encoding, which use fixed block structure of relatively large size. Persons skilled in the art, upon reviewing the entirety of this disclosure, will appreciate that, while for the sake of brevity 4×4 sample blocks are described in many ensuing examples, in general any size or shape of sample blocks, according to any method of measurement, may be used for picture partitioning and/or segmentation.

[0024] Referring now to FIG. 1, an exemplary embodiment of a current frame divided into a plurality of sub-pictures is illustrated. Sub-pictures may include any portion of current frame smaller than current frame; sub-pictures of current frame may combine to cover all of current frame. Although FIG. 1 illustrates exemplary current frames divided into two or four sub-pictures, persons skilled in the art having viewed the entirety of this disclosure will appreciate that any number of sub-pictures may be used as appropriate for resolution, efficiency, or any other consideration.

[0025] Still referring to FIG. 1, a sub-picture may have any suitable shape, including without limitation a square and/or rectangular shape, a shape defined by combination of two or more blocks having square and/or rectangular shapes, or the like. Each block may be identified and/or signaled using coordinates of one or more portions and/or features of a block, where coordinates may indicate number of pixels across frame and/or picture as measured from one or more corners and/or sides of the frame and/or picture. For instance, and without limitation, a block may be identified using coordinates of vertices, such as two x coordinates and two y coordinates for identification of a rectangular block. A sub-picture and/or portion thereof may alternatively or additionally be identified using any suitable geometric description of points, lines, and/or shapes, including without limitation geometric partition using one or more line segments, as defined by linear equations or mathematically equivalent expressions such as line-segment endpoints, using one or more curved edges such as without limitation defined using exponential or other curves, or the like

[0026] With continued reference to FIG. 1, sub-pictures may be coded separately from one another.

For instance, and without limitation, a first region of a plurality of sub-pictures may be encoded and/or decoded using a first processor thread and a second region element may be decoded using a second processor thread. A “processor thread” as used herein may include any processor core and/or other hardware element capable of executing a thread of a multithreaded parallel process that may occur to persons skilled in the art upon reviewing the entirety of this disclosure. In an embodiment, where each sub-picture is independently coded, this form of task partitioning may allow for efficient encoding by using all available computing resources effectively.

[0027] Still referring to FIG. 1, lossless coding may be selectively applied to a subset of blocks of a picture where it is desirable for one or more reasons described above for a source video to be preserved without any loss. As a non-limiting example, selection of a subset of a picture for lossless coding may be done for reasons of coding efficiency. In such cases, lossless coding mode decision may be made after evaluating a rate-distortion (RD) cost of coding a CTU in lossy and lossless modes. In certain use cases, portions of a video may be selected by the user to be encoded in lossless mode for reasons dictated by applications. A non-limiting example may include situations where portion of a frame where source quality retention is desirable for a user. When such user selections are made, an entire region may be marked as using lossless coding without performing any RD analysis.

[0028] Alternatively or additionally, and further referring to FIG. 1, a sub-picture may be identified by an encoder and/or other hardware and/or software component and/or process as an area, region and/or subdivision of a picture in which greater amounts of motion are detected and/or present; such regions may be identified as sub-pictures can considered significant and coded using lossless coding, while sub-pictures with little or no motion may be coded using lossy coding. An example is shown in FIG. 2, where a picture 200 is divided in two sub-pictures: a first region 204 with motion, and second and/or third region 208 with no motion. As noted above, in some cases lossless coding may give better compression than lossy coding with that uses transform and quantization. In an alternative or additional example, a picture 200 may be divided into a first region 204 encoded using a first lossless protocol, a second region (not shown) using a second lossless protocol, and a third region using a lossy protocol.

[0029] Referring again to FIG. 1, a picture may be divided into sub-pictures, slices, and tiles. Blocks (CTUs) may be coding units that may be coded in intra or inter coding mode. A sub-picture may include a single CTU and/or plurality of CTUs. In an embodiment, each CTU in a subset of CTUs may signal whether lossless coding is used in the CTU; alternatively or additionally, a set of CTUs, such as without limitation a set of contiguously located CTUs may be signaled together. Lossless and/or lossy coding may be signaled in one or more headers provided to a bitstream. For instance, and without limitation, CTUs may be coded in lossless coding mode by signaling lossless and/or lossy coding mode in a CTU header. Selective use of lossless coding for a sub-set of blocks (CTUs) may alternatively or additionally be signaled at a higher-level syntactic unit. For example, a tile, slice, and/or sub-picture header may signal the use of lossless coding modes for all the CTUs in that syntactic unit. A sub-picture header may be either explicitly present or included by reference using a mechanism such as an identifier of another header such as a previously signaled picture header.

[0030] As a non-limiting example, and continuing to refer to FIG. 1, data and/or logic within a sub-picture header, CTU header, and/or other header may include, without limitation, a first bit indicating whether lossless mode signaling is enabled, or in other words whether encoder and/or decoder should signal and/or receive a signal indicating whether lossless and/or lossy mode is being used for the relevant CTU, sub-picture, or the like. Data and/or logic within a sub-picture header, CTU header, and/or other header may include, without limitation, a second bit indicating lossless and/or lossy mode, where a lossless mode is a mode in which relevant CTU, sub-picture, or the like is encoded and decoded using a lossless encoding and decoding protocol as described above. The following is a non-limiting and illustrative example of logic and data that may be

employed

TABLE-US-00001 Sub_picture_header{ lossless_mode_signaling [1-bit]

if(lossless_mode_signaling){ lossless_mode [1 bit]; } }

[0031] Still referring to FIG. 1, an encoder and/or decoder configured to perform processes described in this disclosure may be configured to signal and/or detect a lossless encoding protocol used, for instance using an identifier and/or bit corresponding to the lossless encoding protocol. Alternatively or additionally, encoder and/or decoder may be configured to operate a specific lossless encoding and decoding protocol, for instance as consistent with a given standard, release, or other approach to adopting uniform standard. There may be two or more standard protocols, selection of which may be signaled in a bitstream using a sufficient number of bits to encode the two or more potential selections.

[0032] With continued reference to FIG. 1, lossless coding protocol may include any protocol for lossless encoding of images, videos, frames, pictures, sub-pictures or the like. As a non-limiting example, encoder and/or decoder may accomplish lossless coding is to bypass a transform coding stage and encode residual directly. This approach, which may be referred to in this disclosure as “transform skip residual coding,” may be accomplished by skipping transformation of a residual, as described in further detail below, from spatial into frequency domain by applying a transform from the family of discrete cosine transforms (DCTs), as performed for instance in some forms of block-based hybrid video coding. Lossless encoding and decoding may be performed according to one or more alternative processes and/or protocols, including without limitation processes and/or protocols as proposed at Core Experiment CE3-1 of JVET-Q00069 pertaining to regular and TS residual coding (RRC, TSRC) for lossless coding, and modifications to RRC and TSRC for lossless and lossy operation modes, Core Experiment CE3-2 of JVET-Q0080, pertaining to enabling block differential pulse-code modulation (BDPCM) and high-level techniques for lossless coding, and the combination of BDPCM with different RRC/TSRC techniques, or the like.

[0033] With further reference to FIG. 1, an encoder as described in this disclosure may be configured to encode one or more fields using TS residual coding, where one or more fields may include without limitation any picture, sub-picture, coding unit, coding tree unit, tree unit, block, slice, tile, and/or any combination thereof. A decoder as described in this disclosure may be configured to decode one or more fields according to and/or using TS residual coding. In transform skip mode, residuals of a field may be coded in units of non-overlapped subblocks, or other subdivisions, of a given size, such as without limitation a size of four pixels by four pixels. A quantization index of each scan position in a field to be transformed may be coded, instead of coding a last significant scan position; a final subblock and/or subdivision position may be inferred based on levels of previous subdivisions. TS residual coding may perform diagonal scan in a forward manner rather than a reverse manner. Forward scanning order may be applied to scan subblocks within a transform block as well as positions within a subblock and/or subdivision; in an embodiment, there may be no signaling of a final (x, y) position. As a non-limiting example, a coded_sub_block_flag may be coded for every subblock except for a final subblock when all previous flags are equal to 0. sig_coeff_flag context modelling may use a reduced template. A context model of sig_coeff_flag may depend on top and left neighboring values; context model of abs_level_gt1 flag may also depend on left and top sig_coeff_flag values.

[0034] Still referring to FIG. 1, and as a non-limiting example, during a first scan pass in a TS residual coding process, a significance flag (sig_coeff_flag), sign flag (coeff_sign_flag), absolute level greater than 1 flag (abs_level_gtx_flag[0]), and parity (par_level_flag) may be coded. For a given scan position, if sig_coeff_flag is equal to 1, then coeff_sign_flag may be coded, followed by the abs_level_gtx_flag[0] (which specifies whether the absolute level is greater than 1). If abs_level_gtx_flag[0] is equal to 1, then the par_level_flag is additionally coded to specify the parity of the absolute level. During a second or subsequent scan pass, for each scan position whose absolute level is greater than 1, up to four abs_level_gtx_flag[i] for i=1 . . . 4 may be coded to

indicate if an absolute level at a given position is greater than 3, 5, 7, or 9, respectively. During a third or final “remainder” scan pass, remainder, which may be stored as absolute level `abs_remainder` may be coded in a bypass mode. Remainder of absolute levels may be binarized using a fixed rice parameter value of 1.

[0035] Further referring to FIG. 1, bins in a first scan pass and second or “greater-than-x” scan pass may be context coded until a maximum number of context coded bins in a field, such as without limitation a TU, have been exhausted. a maximum number of context coded bins in a residual block may be limited, in a non-limiting example, to $1.75 \times \text{block_width} \times \text{block_height}$, or equivalently, 1.75 context coded bins per sample position on average. Bins in a last scan pass such as a remainder scan pass as described above, may be bypass coded. A variable, such as without limitation `RemCcbs`, may be first set to a maximum number of context-coded bins for a block or other field and may be decreased by one each time a context-coded bin is coded. In a non-limiting example, while `RemCcbs` is larger than or equal to four, syntax elements in a first coding pass, which may include `sig_coeff_flag`, `coeff_sign_flag`, `abs_level_gt1_flag` and `par_level_flag`, may be coded using context-coded bins. In some embodiments, if `RemCcbs` becomes smaller than 4 while coding a first pass, a remaining coefficients that have yet to be coded in the first pass may be coded in the remainder scan pass and/or third pass.

[0036] Still referring to FIG. 1, after completion of first pass coding, if `RemCcbs` is larger than or equal to four, syntax elements in second coding pass, which may include `abs_level_gt3_flag`, `abs_level_gt5_flag`, `abs_level_gt7_flag`, and `abs_level_gt9_flag`, may be coded using context coded bins. If the `RemCcbs` becomes smaller than 4 while coding a second pass, remaining coefficients that have yet to be coded in the second pass may be coded in a remainder and/or third scan pass. In some embodiments, a block coded using TS residual coding may not be coded using BDPCM coding. For a block not coded in the BDPCM mode, a level mapping mechanism may be applied to transform skip residual coding until a maximum number of context coded bins has been reached. Level mapping may use top and left neighboring coefficient levels to predict a current coefficient level in order to reduce signaling cost. For a given residual position, `absCoeff` may be denoted as an absolute coefficient level before mapping and `absCoeffMod` may be denoted as a coefficient level after mapping. As a non-limiting example, where `X0` denotes an absolute coefficient level of a left neighboring position and `X1` denotes an absolute coefficient level of an above neighboring position, a level mapping may be performed as follows:

TABLE-US-00002 $\text{pred} = \max(\text{X.sub.0}, \text{X.sub.1})$; if (`absCoeff` == `pred`) `absCoeffMod` = 1; else `absCoeffMod`

$\text{absCoeffMod} = (\text{absCoeffMod} < \text{pred}) ? \text{absCoeff} + 1 : \text{absCoeff}$
`absCoeffMod` value may then be coded as described above. After all context coded bins have been exhausted, level mapping may be disabled for all remaining scan positions in a current block and/or field and/or subdivision. Three scan passes as described above may be performed for each subblock and/or other subdivision if a `coded_subblock_flag` is equal to 1, which may indicate that there is at least one non-zero quantized residual in the subblock.

[0037] In some embodiments, and still referring to FIG. 1, when transform skip mode is used for a large block, the entire block may be used without zeroing out any values. In addition, transform shift may be removed in transform skip mode. Statistical characteristics of a signal in TS residual coding may be different from those of transform coefficients. Residual coding for transform skip mode may specify a maximum luma and/or chroma block size; as a non-limiting example, settings may permit transform skip mode to be used for luma blocks of size up to `MaxTsSize` by `MaxTsSize`, where a value of `MaxTsSize` may be signaled in a PPS and may have a global maximum possible value such as without limitation 32. When a CU is coded in transform skip mode, its prediction residual may be quantized and coded using a transform skip residual coding process.

[0038] With continued reference to FIG. 1, an encoder as described in this disclosure may be configured to encode one or more fields using BDPCM, where one or more fields may include

without limitation any picture, sub-picture, coding unit, coding tree unit, block, slice, tile, and/or any combination thereof. A decoder as described in this disclosure may be configured to decode one or more fields according to and/or using BDPCM. BDPCM may keep full reconstruction at a pixel level. As a non-limiting example, a prediction process of each pixel with BDPCM may include four main steps, which may predict each pixel using its in-block references, then reconstruct it to be used as in-block reference for subsequent pixels in the rest of the block: (1) in-block pixel prediction, (2) residual calculation, (3) residual quantization, and (4) pixel reconstruction.

[0039] Still referring to FIG. 1, in-block pixel prediction may use a plurality of reference pixels to predict each pixel; as a non-limiting example, plurality of reference pixels may include a pixel α at left of the pixel p to be predicted, a pixel β above p , and a pixel γ above and to the left of p . A prediction of p may be formulated, without limitation, as follows:

TABLE-US-00003
$$p = \begin{cases} \min(\alpha, \beta), & \text{if } \gamma \text{ is available} \\ \max(\alpha, \beta), & \text{if } \gamma \text{ is not available} \\ \alpha + \beta - \gamma, & \text{Otherwise} \end{cases}$$

[0040] Still referring to FIG. 1, once a prediction value has been calculated, its residual may be calculated. Since a residual at this stage may be lossless and inaccessible at a decoder side, it may be denoted as $\hat{r}$ and calculated as a subtraction of an original pixel value o from prediction p :

[00001] $\hat{r} = o - p$

[0041] Further referring to FIG. 1, pixel-level independence may be achieved by skipping a residual transformation and integrating a spatial domain quantization. This may be performed by a linear quantizer Q to calculate a quantized residual value r as follows:

$r = Q(\hat{r})$

To accommodate a correct rate-distortion ratio, imposed by a Quantizer Parameter (QP), BDPCM may adopt a spatial domain normalization used in a transfer-skip mode method, for instance and without limitation as described above. Quantized residual value r may be transmitted by an encoder.

[0042] Still referring to FIG. 1, another state of BDPCM may include pixel reconstruction using p and r from previous steps, which may be performed, for instance and without limitation at or by a decoder, as follows:

[00002] $c = p + r$

Once reconstructed, current pixel may be used as an in-block reference for other pixels within the same block.

[0043] A prediction scheme in an BDPCM algorithm may be used where there is a relatively large residual, when an original pixel value is far from its prediction. In screen content, this may occur where in-block references belong to a background layer, while a current pixel belongs to a foreground layer, or vice versa. In this situation, which may be referred to as a “layer transition” situation, available information in references may not be adequate for an accurate prediction. At a sequence level, a BDPCM enable flag may be signaled in an SPS; this flag may, without limitation, be signaled only if a transform skip mode, for instance and without limitation as described above, is enabled in the SPS. When BDPCM is enabled, a flag may be transmitted at a CU level if a CU size is smaller than or equal to $MaxTsSize$ by $MaxTsSize$ in terms of luma samples and if the CU is intra coded, where $MaxTsSize$ is a maximum block size for which a transform skip mode is allowed. This flag may indicate whether regular intra coding or BDPCM is used. If BDPCM is used, a BDPCM prediction direction flag may be transmitted to indicate whether a prediction is horizontal or vertical. Then, a block may be predicted using regular horizontal or vertical intra prediction process with unfiltered reference samples.

[0044] Referring now to FIG. 3, an exemplary embodiment of a method 300 of combined lossless and lossy coding is illustrated. At step 305, a decoder receives a bitstream. At step 310, decoder identifies a current frame in bitstream. Current frame may include a first region, a second region,

and a third region, any of which may include any region as described above; regions may be flagged using frame header information and/or delineated or otherwise described using coordinates, geometric information, identifications of blocks and/or CTUs included in each region, or the like. In an embodiment, decoder may identify only two regions of first region, second region, and third region in current frame, while a remaining region may be identified as remaining tiles, slices, blocks, CTUs, or the like of current frame. There may be more than three regions; method **300** may include any processing step as described in this disclosure being performed with regard to any additional regions.

[0045] At step **315**, and with continued reference to FIG. 3, decoder detects, in the bitstream, an indication that the first region is encoded according to block differential pulse code modulation; this may be performed, without limitation, as described above in reference to FIGS. 1-2. Detection may include and/or be preceded by detection that block differential pulse code modulation is enabled, for instance as described above. In an embodiment, bitstream may include a sub-picture header corresponding to first region. Detection may include detecting indication that at least a first region is encoded according to block differential pulse code modulation in a sub-picture and/or region-specific header. Sub-picture header may be explicitly included in data corresponding to current frame. For instance, and without limitation, `sps_bdpcm_enabled_flag` may be set to 1 in an SPS and/or other header if bdpcm is enabled for a sequence. `sps_bdpcm_enabled_flag` equal to 1 may specify that an `intra_bdpcm_luma_flag` and/or an `intra_bdpcm_chroma_flag` may be present in coding unit and/or other field-specific syntax for intra coding units and/or other fields. `sps_bdpcm_enabled_flag` equal to 0 may specify that `intra_bdpcm_luma_flag` and/or `intra_bdpcm_chroma_flag` are not present in coding unit and/or other field-specific syntax for intra coding units and/or other fields. When not present, a value of `sps_bdpcm_enabled_flag` may be inferred to be equal to 0. In an embodiment, a `gci_no_bdpcm_constraint_flag` equal to 1 may specify that `sps_bdpcm_enabled_flag` for all pictures in a given set, which may be defined without limitation by an `OlsInScope` parameter, shall be equal to 0. `gci_no_bdpcm_constraint_flag` equal to 0 may not impose such a constraint. As a further non-limiting example, `intra_bdpcm_luma_flag` equal to 1 may specify that BDPCM may be applied to a current luma coding block, and/or other field at a location (x0, y0), i.e. the transform is skipped; a luma intra prediction mode may be specified by `intra_bdpcm_luma_dir_flag`. For instance and without limitation, `intra_bdpcm_luma_flag` equal to 0 may specify that BDPCM is not applied to a current luma coding block, and/or other field, at a location (x0, y0). When `intra_bdpcm_luma_flag` is not present it may be inferred to be equal to 0. A variable `BdpcmFlag[x][y][cIdx]` may be set equal to `intra_bdpcm_luma_flag` for `x=x0 . . . x0+cbWidth-1`, `y=y0 . . . y0+cbHeight-1` and `cIdx=0`. `intra_bdpcm_luma_dir_flag` equal to 0 may specify that a BDPCM prediction direction is horizontal. `intra_bdpcm_luma_dir_flag` equal to 1 may specify that a BDPCM prediction direction is vertical. Variable `BdpcmDir[x][y][cIdx]` may be set equal to `intra_bdpcm_luma_dir_flag` for `x=x0..x0+cbWidth-1`, `y=y0..y0+cbHeight-1` and `cIdx=0`. Sub-picture and/or region may be included by reference to an identifier of a sub-picture header corresponding to a third sub-picture and/or other element of current frame.

[0046] At step **320**, and with continued reference to FIG. 3, decoder detects, in the bitstream, an indication that second region is encoded according to transfer skip residual coding; this may be performed, without limitation, as described above in reference to FIGS. 1-2. Detection may include and/or be preceded by detection that transfer skip residual coding is enabled, for instance as described above. In an embodiment, bitstream may include a sub-picture header corresponding to first region. Detection may include detecting indication that at least a first region is encoded according to a transfer skip residual coding protocol in a sub-picture header; this may include a transform skip enable flag. For instance, and without limitation, a `sh_ts_residual_coding_disabled_flag` equal to 1 may specify that a `residual_coding` syntax structure may be used to parse the residual samples of a transform skip block for a current slice and/or other

field. `sh_ts_residual_coding_disabled_flag` equal to 0 may specify that a `residual_ts_coding` syntax structure may be used to parse residual samples of a transform skip block for a current slice. When `sh_ts_residual_coding_disabled_flag` is not present, it may be inferred to be equal to 0.

`transform_skip_flag[x0][y0][cIdx]` may specify whether a transform may be applied to an associated transform block or not. Array indices `x0`, `y0` may specify a location (`x0`, `y0`) of a top-left luma sample of a considered transform block relative to a top-left luma sample of a picture. An array index `cIdx` may specify an indicator for a colour component; it may, for instance, be equal to 0 for Y, 1 for Cb, and 2 for Cr. `transform_skip_flag[x0][y0][cIdx]` equal to 1 may specify that no transform may be applied to an associated transform block. `transform_skip_flag[x0][y0][cIdx]` equal to 0 may specify that a decision whether transform is applied to the associated transform block or not depends on other syntax elements. Transform skip mode may alternatively or additionally be signaled implicitly. For instance, When `transform_skip_flag[x0][y0][cIdx]` is not present, it may be inferred as follows: If `BdpcmFlag[x0][y0][cIdx]` is equal to 1 `transform_skip_flag[x0][y0][cIdx]` may be inferred to be equal to 1; otherwise, where `BdpcmFlag[x0][y0][cIdx]` is equal to 0, `transform_skip_flag[x0][y0][cIdx]` may be inferred to be equal to 0.

Sub-picture and/or region-specific header may be explicitly included in data corresponding to current frame. Sub-picture and/or region may be included by reference to an identifier of a sub-picture header corresponding to a third sub-picture and/or other element of current frame.

[0047] At step **325**, and with continued reference to FIG. **3**, decoder detects that third region is encoded according to a lossy encoding protocol decode third region according to a lossy decoding protocol corresponding to the lossless encoding protocol; this may be performed according to any lossy decoding process described herein, including processes including DCT and other processes as described below. Bitstream may include a sub-picture and/or region-specific header corresponding to the third region and detecting may include the indication that the third region is encoded according to a lossy encoding protocol in the sub-picture header. Sub-picture and/or region-specific header may be explicitly included in data corresponding to the current frame. Sub-picture and/or region-specific header may be included by reference to an identifier of a sub-picture and/or region-specific header corresponding to a third sub-picture, for instance as described above. In an embodiment, decoder may be configured to decode first region using a first processor thread, as defined above, and decode third region clement using a second processor thread.

[0048] Continuing to refer to FIG. **3**, decoder may decode current frame. Decoding current frame may include decoding first region using a BDPCM decoding protocol corresponding to BDPCM encoding protocol. Decoding current frame may include decoding second region using a transfer-skip residual decoding protocol corresponding to a transfer-skip residual encoding protocol. Decoding current frame may include decoding third region using a lossy decoding protocol corresponding to lossy encoding protocol.

[0049] Still referring to FIG. **3**, the decoder may include an entropy decoder processor configured to receive the bit stream and decode the bitstream into quantized coefficients, an inverse quantization and inverse transformation processor configured to process the quantized coefficients including performing an inverse discrete cosine, a deblocking filter, a frame buffer, and an intra prediction processor. At least one of first region, second region, and third region may form part of a quadtree plus binary decision tree. At least one of first region, second region, and third region includes a coding tree unit. In some implementations, at least one of first region, second region, and third region may include a coding tree unit (CTU), a coding unit (CU), or a prediction unit (PU).

[0050] FIG. **4** is a system block diagram illustrating an example decoder **400** capable of decoding a bitstream **428** using combined lossy and lossless coding protocols. Decoder **400** may include an entropy decoder processor **404**, an inverse quantization and inverse transformation processor **408**, a deblocking filter **412**, a frame buffer **416**, a motion compensation processor **420** and/or an intra prediction processor **424**.

[0051] In operation, and still referring to FIG. **4**, bit stream **428** may be received by decoder **400**

and input to entropy decoder processor **404**, which may entropy decode portions of bit stream into quantized coefficients. Quantized coefficients may be provided to inverse quantization and inverse transformation processor **408**, which may perform inverse quantization and inverse transformation to create a residual signal, which may be added to an output of motion compensation processor **420** or intra prediction processor **424** according to a processing mode. An output of the motion compensation processor **420** and intra prediction processor **424** may include a block prediction based on a previously decoded block. A sum of prediction and residual may be processed by deblocking filter **412** and stored in a frame buffer **416**.

[0052] With continued reference to FIG. **4** decoder **400** may be designed and/or configured to perform any method, method step, or sequence of method steps in any embodiment described in this disclosure, in any order and with any degree of repetition. For instance, decoder **400** may be configured to perform a single step or sequence repeatedly until a desired or commanded outcome is achieved; repetition of a step or a sequence of steps may be performed iteratively and/or recursively using outputs of previous repetitions as inputs to subsequent repetitions, aggregating inputs and/or outputs of repetitions to produce an aggregate result, reduction or decrement of one or more variables such as global variables, and/or division of a larger processing task into a set of iteratively addressed smaller processing tasks. Decoder **400** may perform any step or sequence of steps as described in this disclosure in parallel, such as simultaneously and/or substantially simultaneously performing a step two or more times using two or more parallel threads, processor cores, or the like; division of tasks between parallel threads and/or processes may be performed according to any protocol suitable for division of tasks between iterations. Persons skilled in the art, upon reviewing the entirety of this disclosure, will be aware of various ways in which steps, sequences of steps, processing tasks, and/or data may be subdivided, shared, or otherwise dealt with using iteration, recursion, and/or parallel processing.

[0053] Referring now to FIG. **5**, an exemplary embodiment of a method of combined lossless and lossy coding is illustrated. At step **505** an encoder receives an input video. This may be accomplished in any manner suitable for receiving a video in streaming and/or file form from any device and/or input port. Receiving video may include retrieval of one or more frames from memory of encoder and/or a computing device in communication with, incorporating, and/or incorporated in encoder. Receiving may include receiving one or more frames a remote device over a network. Receiving video may include receiving a plurality of video frames that combine to make up one or more videos.

[0054] At step **510**, and continuing to refer to FIG. **5**, encoder selects a current frame; this may be implemented, without limitation, any manner suitable for selection of a current frame of one or more frames making up a video. Encoder may segment and/or partition video frame into blocks. The blocks may have any suitable shape or size, including a size of 4 pixels by 4 pixels (4×4). A 4×4 size may be compatible with many standard video resolutions, which may be divided into an integer number of 4×4 blocks.

[0055] At step **515**, and still referring to FIG. **5**, encoder identifies a first region of current frame to be encoded using a lossless encoding protocol. In an embodiment, this may be performed by receiving a user identification of first region, for instance by selection of an area of frame, of a number of blocks of frame, or the like. Alternatively or additionally, identification of first region may be performed by receiving semantic information regarding one or more blocks and/or portions of frame and using semantic information to identify blocks and/or portions of frame for inclusion in first region. Semantic information may include, without limitation data characterizing a facial detection. Facial detection and/or other semantic information may be performed by an automated facial recognition process and/or program, and/or may be performed by receiving identification of facial data, semantic information, or the like from a user. Alternatively or additionally, semantic importance may be computed using significance scores as described below.

[0056] Still referring to FIG. **5**, encoder may identify first region by determining an average

measure of information of a plurality of blocks and identifying the first region using the average measure of information. Identification may include, for instance, comparison of average measure of information to a threshold. Average measure of information may be determined by calculating a sum of a plurality of information measures of the plurality of blocks, which may be multiplied by a significance coefficient. Significance coefficient may be determined based on a characteristic of the first area. Significance coefficient may alternatively be received from a user. Measure of information may include, for example, a level of detail of an area of current frame. For example, a smooth area or a highly textured area may contain differing amounts of information.

[0057] Still referring to FIG. 5, an average measure of information may be determined, as a non-limiting example, according to a sum of information measures for individual blocks within an area, which may be weighted and/or multiplied by a significance coefficient, for instance as shown in the following sum:

$$[00003] A_N = S_N * L_{k \circ 11}^n B_k$$

where N is a sequential number of the first area, S.sub.N is a significance coefficient, k is an index corresponding to a block of a plurality of blocks making up first area, n is a number of blocks making up the area, B.sub.k is a measure of information of a block of the blocks, and A.sub.N is the first average measure of information. B.sub.k may include, for example, a measure of spatial activity computed using a discrete cosine transform of a block. For example, where blocks as described above are 4×4 blocks of pixels, a generalized discrete cosine transform matrix may include a generalized discrete cosine transform II matrix taking the form of:

$$[00004] T = \begin{bmatrix} a & a & a & a \\ b & c & -c & -b \\ -a & -a & a & \\ c & -b & b & -c \end{bmatrix} \text{ where } a \text{ is } 1/2, b \text{ is } J_2^1 \cos \frac{\pi}{8}, \text{ and } c \text{ is } J_2^1 \cos \frac{3\pi}{8}.$$

[0058] In some implementations, and still referring to FIG. 5, an integer approximation of a transform matrix may be utilized, which may be used for efficient hardware and software implementations. For example, where blocks as described above are 4×4 blocks of pixels, a generalized discrete cosine transform matrix may include a generalized discrete cosine transform II matrix taking the form of:

$$[00005] T_{\text{INT}} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 2 & 1 & -1 & -2 \\ 1 & -1 & -1 & -1 \\ 1 & -2 & 2 & -1 \end{bmatrix}$$

[0059] For a block B.sub.i, a frequency content of the block may be calculated using:

$$[00006] F_{Bi} = T \times B_i \times T'$$

where T' is a transverse of a cosine transfer matrix T, B.sub.i is a block represented as a matrix of numerical values corresponding to pixels in the block, such as a 4×4 matrix representing a 4×4 block as described above, and the operation x denotes matrix multiplication. Measure of spatial activity may alternatively or additionally be performed using edge and/or corner detection, convolution with kernels for pattern detection, and/or frequency analysis such as without limitation FFT processes as described in further detail below.

[0060] Continuing to refer to FIG. 5, where encoder is further configured to determine a second area within the video frame as described in further detail below, encoder may be configured to determine a second average measure of information of the second area; determining the second average measure of information may be accomplished as described above for determining a first average measure of information.

[0061] Still referring to FIG. 5, significance coefficient Sx may be supplied by an outside expert and/or calculated based on the characteristics of an area. A “characteristic” of an area, as used herein, is a measurable attribute of the area that is determined based upon its contents; a characteristic may be represented numerically using an output of one or more computations

performed on first area. One or more computations may include any analysis of any signal represented by first area. One non-limiting example may include assigning higher S_x for an area with a smooth background and a lower $S_{sub.N}$ for an area with a less smooth background in quality modeling applications; as a non-limiting example, smoothness may be determined using Canny edge detection to determine a number of edges, where a lower number indicates a greater degree of smoothness. A further example of automatic smoothness detection may include use of fast Fourier transforms (FFT) over a signal in spatial variables over an area, where signal may be analyzed over any two-dimensional coordinate system, and over channels representing red-green-blue color values or the like; greater relative predominance in a frequency domain, as computed using an FFT, of lower frequency components may indicate a greater degree of smoothness, whereas greater relative predominance of higher frequencies may indicate more frequent and rapid transitions in color and/or shade values over background area, which may result in a lower smoothness score; semantically important objects may be identified by user input. Semantic importance may alternatively or additionally be detected according to edge configuration, and/or texture pattern. A background may be identified, without limitation, by receiving and/or detecting a portion of an area that represents significant or “foreground” object such as a face or other item, including without limitation a semantically important object. Another example can include assigning higher S_x for the areas containing semantically important objects, such as human face.

[0062] Further referring to FIG. 5, identifying first region may include determining a measure of spatial activity of each block of a plurality of blocks and identifying the first region using the measure of spatial activity. As used in this disclosure, a “spatial activity measure” is a quantity indicating how frequently and with what amplitude texture changes within a block, set of blocks, and/or area of a frame. In other words, flat areas, such as sky, may have a low spatial activity measure, while complex areas such as grass will receive a high spatial activity measure. Determination of a respective spatial activity measure may include determination using a transform matrix, such without limitation a discrete cosine transformation matrix. Determining respective spatial activity measure for each block may include determination using a generalized discrete cosine transformation matrix, which may include without limitation any discrete cosine transformation matrix as described above. For instance, determining the respective spatial activity measure for each block may include using a generalized discrete cosine transformation matrix, a generalized discrete cosine transform II matrix, and/or an integer approximation of a discrete cosine transform matrix.

[0063] Still referring to FIG. 5, multiple blocks and/or other areas identified for lossless encoding according to any method described in this disclosure may be combined and/or fused together as first region; for instance, and without limitation, a plurality of contiguous blocks so identified may be combined and/or fused together. Block fusion may include determining a first area within the video frame including a first grouping of a first subset of the blocks. In block fusion, each block may be assigned to an area. Assignment logic, such as without limitation semantic information, may be obtained from an outside source, which semantic information may be received at 130; as a non-limiting example, semantic information may include information provided from a face detector such that the semantic information includes data characterizing a facial detection. Accordingly, first grouping may be determined based on the received semantic information. In some implementations, assignment logic may be predefined, for example, according to a number of clustering or grouping algorithms. Encoder may be further configured to determine a second area within video frame including a second grouping of a second subset of the blocks.

[0064] Continuing to refer to FIG. 5, any block that has at least one pixel belonging to an object of interest, such as without limitation a face, as identified for example via received semantic information, may be assigned to a shaded area (A_2), for instance according to the received semantic information, and fused with other blocks in that area.

[0065] In some implementations, and still referring to FIG. 5, block fusion may be performed using

the spatial activity measures. In block fusion, each block may be assigned to an area. For example, a first area within the video frame including a first grouping of a subset of the blocks may be determined. The first grouping may be based on the respective spatial activity measure. Blocks may be grouped into areas of similar frequency content thereby representing areas with similar spatial activity. For example, determining the first area may include iterating over each of the plurality of blocks and, for each current block, comparing the spatial activity measure of the current block to a spatial activity measure of a preceding block and determining whether a difference is below a predefined threshold, and assigning the current block to the first area. The current block may be assigned to a second area in response to or based on determining that the difference is above the predefined threshold. Persons skilled in the art, upon reviewing the entirety of this disclosure, will be aware of various ways in which threshold comparison may be performed, including that assignment to a first block may alternatively or additionally be accomplished by determination that a degree of similarity exceeds a threshold and/or that assignment to a different group may be accomplished by determining that a degree of difference exceeds a threshold. A threshold may be stored in memory and/or generated using previously or concurrently received and/or calculated values, which may include any numerical values and/or measurements described in this disclosure. Lossless encoding protocol for first frame may include any encoding protocol described above, including without limitation BDPCM.

[0066] At step **520**, and continuing to refer to FIG. 5, encoder encodes the current frame, wherein encoding the current frame further comprises encoding the first region using the lossless encoding protocol. Encoding current frame further may include inserting an indication that at least a first region is encoded according to a lossless encoding protocol in a sub-picture header, associating the sub-picture header with the at least a first region, and including the sub-picture header in bitstream; this may be implemented without limitation as described above in reference to FIGS. 1-4.

Including sub-picture header in bitstream further comprises explicitly including the sub-picture header in data corresponding to current frame. Alternatively or additionally, including sub-picture header in bitstream may include the sub-picture header in the bitstream further comprises including a reference to an identifier of a sub-picture header corresponding to a third sub-picture. Encoding may include indicating in a bitstream field a type of lossless encoding protocol.

[0067] In an embodiment, and still referring to FIG. 5, encoder may identify a second region to be encoded according to a lossless encoding protocol and encode the second region according to the lossless encoding protocol. Identification may be performed using any process and/or process step for identification of first region. Lossless encoding protocol may include, without limitation, any encoding protocol described above, including without limitation transform skip residual coding.

[0068] In an embodiment, and still referring to FIG. 5, encoder may identify a third region to be encoded according to a lossy encoding protocol and encode the third region according to the lossy encoding protocol. This may be performed, without limitation, using any form of determination and/or threshold comparison described above, where failure to meet or exceed a threshold indicative of inclusion in first area may indicate inclusion in second area and/or that a lossy encoding protocol is to be employed. Identification may include, for instance, comparison of average measure of information to a threshold, comparison of a measure of spatial activity to a threshold, comparison of a significance score to a threshold, or the like. Identifying third region may include receiving a user identification of third region, which may be performed, without limitation as described above for receiving a user identification of first region.

[0069] With continued reference to FIG. 5, encoder may insert an indication that third region is encoded according to a lossless encoding protocol in a sub-picture header, associate the sub-picture header with the third region, and including the sub-picture header in bitstream; this may be performed as described in further detail above. For instance, and without limitation, encoder may include sub-picture header explicitly in data corresponding to the current frame. Alternatively or additionally, encoder may include sub-picture header by reference to an identifier of a sub-picture

header corresponding to a third sub-picture.

[0070] Still referring to FIG. 5, encoder may use multi-threading and/or parallel processing of sub-pictures, as described above, to improve efficiency of encoding process. For instance, encoder may encode the first region using a first processor thread and encode the third region element using a second processor thread.

[0071] FIG. 6 is a system block diagram illustrating an exemplary embodiment of a video encoder **600** capable of constructing a motion vector candidate list including adding a single global motion vector candidate to the motion vector candidate list. Example video encoder **600** may receive an input video **604**, which may be initially segmented and/or dividing according to a processing scheme, such as a tree-structured macro block partitioning scheme (e.g., quad-tree plus binary tree). An example of a tree-structured macro block partitioning scheme may include partitioning a picture frame into large block elements called coding tree units (CTU). In some implementations, each CTU may be further partitioned one or more times into a number of sub-blocks called coding units (CU). A final result of this portioning may include a group of sub-blocks that may be called predictive units (PU). Transform units (TU) may also be utilized.

[0072] Still referring to FIG. 6, example video encoder **600** may include an intra prediction processor **608**, a motion estimation/compensation processor **612** (also referred to as an inter prediction processor) capable of constructing a motion vector candidate list including adding a single global motion vector candidate to the motion vector candidate list, a transform/quantization processor **616**, an inverse quantization/inverse transform processor **620**, an in-loop filter **624**, a decoded picture buffer **628**, and/or an entropy coding processor **632**. Bit stream parameters may be input to entropy coding processor **632** for inclusion in an output bit stream **636**.

[0073] In operation, and with continued reference to FIG. 6, for each block of a frame of input video **604**, whether to process block via intra picture prediction or using motion estimation/compensation may be determined. Block may be provided to intra prediction processor **608** or motion estimation/compensation processor **612**. If block is to be processed via intra prediction, intra prediction processor **608** may perform processing to output a predictor. If block is to be processed via motion estimation/compensation, motion estimation/compensation processor **612** may perform processing including constructing a motion vector candidate list including adding a single global motion vector candidate to the motion vector candidate list, if applicable.

[0074] Still referring to FIG. 6, a residual may be formed by subtracting predictor from input video. Residual may be received by transform/quantization processor **616**, which may perform transformation processing (e.g., discrete cosine transform (DCT)) to produce coefficients, which may be quantized. Quantized coefficients and any associated signaling information may be provided to entropy coding processor **632** for entropy encoding and inclusion in an output bit stream **636**. Entropy encoding processor **632** may support encoding of signaling information related to encoding a current block. In addition, quantized coefficients may be provided to inverse quantization/inverse transformation processor **620**, which may reproduce pixels, which may be combined with predictor and processed by in loop filter **624**, an output of which may be stored in decoded picture buffer **628** for use by motion estimation/compensation processor **612** that is capable of constructing a motion vector candidate list including adding a single global motion vector candidate to the motion vector candidate list.

[0075] Further referencing FIG. 6, although a few variations have been described in detail above, other modifications or additions are possible. For example, in some implementations, current blocks may include any symmetric blocks (8×8, 16×16, 32×32, 64×64, 128×128, and the like) as well as any asymmetric block (8×4, 16×8, and the like).

[0076] In some implementations, and still referring to FIG. 6, a quadtree plus binary decision tree (QTBT) may be implemented. In QTBT, at a Coding Tree Unit level, partition parameters of QTBT may be dynamically derived to adapt to local characteristics without transmitting any overhead. Subsequently, at a Coding Unit level, a joint-classifier decision tree structure may eliminate

unnecessary iterations and control risk of false prediction. In some implementations, LTR frame block update mode may be available as an additional option available at every leaf node of a QTBT.

[0077] In some implementations, and still referring to FIG. 6, additional syntax elements may be signaled at different hierarchy levels of a bitstream. For example, a flag may be enabled for an entire sequence by including an enable flag coded in a Sequence Parameter Set (SPS). Further, a CTU flag may be coded at a coding tree unit (CTU) level.

[0078] With continued reference to FIG. 6, non-transitory computer program products (i.e., physically embodied computer program products) may store instructions, which when executed by one or more data processors of one or more computing systems, causes at least one data processor to perform operations, and/or steps thereof described in this disclosure, including without limitation any operations described above **400** and/or encoder **600** may be configured to perform. Similarly, computer systems are also described that may include one or more data processors and memory coupled to the one or more data processors. The memory may temporarily or permanently store instructions that cause at least one processor to perform one or more of the operations described herein. In addition, methods can be implemented by one or more data processors either within a single computing system or distributed among two or more computing systems. Such computing systems can be connected and can exchange data and/or commands or other instructions or the like via one or more connections, including a connection over a network (e.g. the Internet, a wireless wide area network, a local area network, a wide area network, a wired network, or the like), via a direct connection between one or more of the multiple computing systems, or the like.

[0079] With continued reference to FIG. 5, encoder **500**, decoder **400**, and/or circuitry thereof may be designed and/or configured to perform any method, method step, or sequence of method steps in any embodiment described in this disclosure, in any order and with any degree of repetition. For instance, encoder **500**, decoder **400**, and/or circuitry thereof may be configured to perform a single step or sequence repeatedly until a desired or commanded outcome is achieved; repetition of a step or a sequence of steps may be performed iteratively and/or recursively using outputs of previous repetitions as inputs to subsequent repetitions, aggregating inputs and/or outputs of repetitions to produce an aggregate result, reduction or decrement of one or more variables such as global variables, and/or division of a larger processing task into a set of iteratively addressed smaller processing tasks. Encoder **500**, decoder **400**, and/or circuitry thereof may perform any step or sequence of steps as described in this disclosure in parallel, such as simultaneously and/or substantially simultaneously performing a step two or more times using two or more parallel threads, processor cores, or the like; division of tasks between parallel threads and/or processes may be performed according to any protocol suitable for division of tasks between iterations. Persons skilled in the art, upon reviewing the entirety of this disclosure, will be aware of various ways in which steps, sequences of steps, processing tasks, and/or data may be subdivided, shared, or otherwise dealt with using iteration, recursion, and/or parallel processing.

[0080] It is to be noted that any one or more of the aspects and embodiments described herein may be conveniently implemented using one or more machines (e.g., one or more computing devices that are utilized as a user computing device for an electronic document, one or more server devices, such as a document server, etc.) programmed according to the teachings of the present specification, as will be apparent to those of ordinary skill in the computer art. Appropriate software coding can readily be prepared by skilled programmers based on the teachings of the present disclosure, as will be apparent to those of ordinary skill in the software art. Aspects and implementations discussed above employing software and/or software modules may also include appropriate hardware for assisting in the implementation of the machine executable instructions of the software and/or software module.

[0081] Such software may be a computer program product that employs a machine-readable storage medium. A machine-readable storage medium may be any medium that is capable of

storing and/or encoding a sequence of instructions for execution by a machine (e.g., a computing device) and that causes the machine to perform any one of the methodologies and/or embodiments described herein. Examples of a machine-readable storage medium include, but are not limited to, a magnetic disk, an optical disc (e.g., CD, CD-R, DVD, DVD-R, etc.), a magneto-optical disk, a read-only memory “ROM” device, a random-access memory “RAM” device, a magnetic card, an optical card, a solid-state memory device, an EPROM, an EEPROM, and any combinations thereof. A machine-readable medium, as used herein, is intended to include a single medium as well as a collection of physically separate media, such as, for example, a collection of compact discs or one or more hard disk drives in combination with a computer memory. As used herein, a machine-readable storage medium does not include transitory forms of signal transmission.

[0082] Such software may also include information (e.g., data) carried as a data signal on a data carrier, such as a carrier wave. For example, machine-executable information may be included as a data-carrying signal embodied in a data carrier in which the signal encodes a sequence of instruction, or portion thereof, for execution by a machine (e.g., a computing device) and any related information (e.g., data structures and data) that causes the machine to perform any one of the methodologies and/or embodiments described herein.

[0083] Examples of a computing device include, but are not limited to, an electronic book reading device, a computer workstation, a terminal computer, a server computer, a handheld device (e.g., a tablet computer, a smartphone, etc.), a web appliance, a network router, a network switch, a network bridge, any machine capable of executing a sequence of instructions that specify an action to be taken by that machine, and any combinations thereof. In one example, a computing device may include and/or be included in a kiosk.

[0084] FIG. 7 shows a diagrammatic representation of one embodiment of a computing device in the exemplary form of a computer system **700** within which a set of instructions for causing a control system to perform any one or more of the aspects and/or methodologies of the present disclosure may be executed. It is also contemplated that multiple computing devices may be utilized to implement a specially configured set of instructions for causing one or more of the devices to perform any one or more of the aspects and/or methodologies of the present disclosure. Computer system **700** includes a processor **704** and a memory **708** that communicate with each other, and with other components, via a bus **712**. Bus **712** may include any of several types of bus structures including, but not limited to, a memory bus, a memory controller, a peripheral bus, a local bus, and any combinations thereof, using any of a variety of bus architectures.

[0085] Processor **704** may include any suitable processor, such as without limitation a processor incorporating logical circuitry for performing arithmetic and logical operations, such as an arithmetic and logic unit (ALU), which may be regulated with a state machine and directed by operational inputs from memory and/or sensors; processor **704** may be organized according to Von Neumann and/or Harvard architecture as a non-limiting example. Processor **704** may include, incorporate, and/or be incorporated in, without limitation, a microcontroller, microprocessor, digital signal processor (DSP), Field Programmable Gate Array (FPGA), Complex Programmable Logic Device (CPLD), Graphical Processing Unit (GPU), general purpose GPU, Tensor Processing Unit (TPU), analog or mixed signal processor, Trusted Platform Module (TPM), a floating-point unit (FPU), and/or system on a chip (SoC).

[0086] Memory **708** may include various components (e.g., machine-readable media) including, but not limited to, a random-access memory component, a read only component, and any combinations thereof. In one example, a basic input/output system **716** (BIOS), including basic routines that help to transfer information between elements within computer system **700**, such as during start-up, may be stored in memory **708**. Memory **708** may also include (e.g., stored on one or more machine-readable media) instructions (e.g., software) **720** embodying any one or more of the aspects and/or methodologies of the present disclosure. In another example, memory **708** may further include any number of program modules including, but not limited to, an operating system,

one or more application programs, other program modules, program data, and any combinations thereof.

[0087] Computer system **700** may also include a storage device **724**. Examples of a storage device (e.g., storage device **724**) include, but are not limited to, a hard disk drive, a magnetic disk drive, an optical disc drive in combination with an optical medium, a solid-state memory device, and any combinations thereof. Storage device **724** may be connected to bus **712** by an appropriate interface (not shown). Example interfaces include, but are not limited to, SCSI, advanced technology attachment (ATA), serial ATA, universal serial bus (USB), IEEE 1394 (FIREWIRE), and any combinations thereof. In one example, storage device **724** (or one or more components thereof) may be removably interfaced with computer system **700** (e.g., via an external port connector (not shown)). Particularly, storage device **724** and an associated machine-readable medium **728** may provide nonvolatile and/or volatile storage of machine-readable instructions, data structures, program modules, and/or other data for computer system **700**. In one example, software **720** may reside, completely or partially, within machine-readable medium **728**. In another example, software **720** may reside, completely or partially, within processor **704**.

[0088] Computer system **700** may also include an input device **732**. In one example, a user of computer system **700** may enter commands and/or other information into computer system **700** via input device **732**. Examples of an input device **732** include, but are not limited to, an alpha-numeric input device (e.g., a keyboard), a pointing device, a joystick, a gamepad, an audio input device (e.g., a microphone, a voice response system, etc.), a cursor control device (e.g., a mouse), a touchpad, an optical scanner, a video capture device (e.g., a still camera, a video camera), a touchscreen, and any combinations thereof. Input device **732** may be interfaced to bus **712** via any of a variety of interfaces (not shown) including, but not limited to, a serial interface, a parallel interface, a game port, a USB interface, a FIREWIRE interface, a direct interface to bus **712**, and any combinations thereof. Input device **732** may include a touch screen interface that may be a part of or separate from display **736**, discussed further below. Input device **732** may be utilized as a user selection device for selecting one or more graphical representations in a graphical interface as described above.

[0089] A user may also input commands and/or other information to computer system **700** via storage device **724** (e.g., a removable disk drive, a flash drive, etc.) and/or network interface device **740**. A network interface device, such as network interface device **740**, may be utilized for connecting computer system **700** to one or more of a variety of networks, such as network **744**, and one or more remote devices **748** connected thereto. Examples of a network interface device include, but are not limited to, a network interface card (e.g., a mobile network interface card, a LAN card), a modem, and any combination thereof. Examples of a network include, but are not limited to, a wide area network (e.g., the Internet, an enterprise network), a local area network (e.g., a network associated with an office, a building, a campus or other relatively small geographic space), a telephone network, a data network associated with a telephone/voice provider (e.g., a mobile communications provider data and/or voice network), a direct connection between two computing devices, and any combinations thereof. A network, such as network **744**, may employ a wired and/or a wireless mode of communication. In general, any network topology may be used. Information (e.g., data, software **720**, etc.) may be communicated to and/or from computer system **700** via network interface device **740**.

[0090] Computer system **700** may further include a video display adapter **752** for communicating a displayable image to a display device, such as display device **736**. Examples of a display device include, but are not limited to, a liquid crystal display (LCD), a cathode ray tube (CRT), a plasma display, a light emitting diode (LED) display, and any combinations thereof. Display adapter **752** and display device **736** may be utilized in combination with processor **704** to provide graphical representations of aspects of the present disclosure. In addition to a display device, computer system **700** may include one or more other peripheral output devices including, but not limited to,

an audio speaker, a printer, and any combinations thereof. Such peripheral output devices may be connected to bus 712 via a peripheral interface 756. Examples of a peripheral interface include, but are not limited to, a serial port, a USB connection, a FIREWIRE connection, a parallel connection, and any combinations thereof.

[0091] The foregoing has been a detailed description of illustrative embodiments of the invention. Various modifications and additions can be made without departing from the spirit and scope of this invention. Features of each of the various embodiments described above may be combined with features of other described embodiments as appropriate in order to provide a multiplicity of feature combinations in associated new embodiments. Furthermore, while the foregoing describes a number of separate embodiments, what has been described herein is merely illustrative of the application of the principles of the present invention. Additionally, although particular methods herein may be illustrated and/or described as being performed in a specific order, the ordering is highly variable within ordinary skill to achieve methods, systems, and software according to the present disclosure. Accordingly, this description is meant to be taken only by way of example, and not to otherwise limit the scope of this invention.

[0092] Exemplary embodiments have been disclosed above and illustrated in the accompanying drawings. It will be understood by those skilled in the art that various changes, omissions and additions may be made to that which is specifically disclosed herein without departing from the spirit and scope of the present invention.

Claims

1. A decoder configured to: receive a bitstream, the bitstream including a coded picture, the coded picture including a first coded sub-picture and a second coded sub-picture, the bitstream also including a first sub-picture header and a second sub-picture header; detect in the first sub-picture header that the first sub-picture is coded using a lossless encoding protocol; detect in the second sub-picture header that the second sub-picture is coded using a lossy encoding protocol; decode the first sub-picture using a protocol for lossless decoding corresponding to the detected lossless encoding protocol; and decode the second sub-picture using a protocol corresponding to the detected lossy encoding protocol.
 2. The decoder of claim 1 wherein the first sub-picture is an independently coded sub-picture.
 3. The decoder of claim 2 wherein the second sub-picture is an independently coded sub-picture.
 4. The decoder of claim 1 wherein the coded picture includes a region coded using transform skip residual coding.
 5. The decoder of claim 4 wherein the region coded using transform skip residual coding is the first sub-picture.
 6. The decoder of claim 1 wherein the coded picture further includes a region coded using BDPCM.
 7. The decoder of claim 1 wherein the picture includes an intra-coded region.
-